

# Proceso Unificado de Desarrollo (PUD)

El PUD es guía que marca los lineamientos generales para transformar los requerimientos del cliente en un sistema de información funcional, sin importar la implementación, la tecnología o aspectos particulares.

El PUD se basa en los casos de uso (que son fragmentos de funcionalidad del sistema que brindan un resultado de valor para el actor) y en la arquitectura que es como se deberán llevar a cabo esos casos de uso. Es iterativo e incremental mediante las fases (inicio, elaboración, construcción y transición) donde en cada una de estas se aplican todos los workflow (requerimientos, análisis, diseño, implementación y prueba) en diferente proporción dependiendo de la fase.

- **Los workflow:** Son actividades secuenciales y lógicas realizadas por los trabajadores que dan como resultado de valor un artefacto.
- **Las vistas:** Son una proyección o perspectiva de la arquitectura del sistema, su función es brindar una forma de observar al sistema desde un ángulo en particular.
- **Asociación con modelos:** Cada workflow está asociado a con uno o mas modelos. Los modelos a su vez cuentan con herramientas que son diagramas que ofrece UML para representar la realidad (para construir los artefactos del workflow).

## Clasificación de Diagramas UML

Tenemos 2 tipos de diagramas:

### 1. Modelan estructura

- **Modelan objetos:** \* Diagrama de objetos.
- **Modelan clases:** \* Diagrama de clases.
  - Diagrama de componentes.
  - Diagrama de paquetes.
  - Diagrama de artefactos.
  - Diagrama de secuencia.
  - Diagrama de estructura compuesta.

### 2. Modelan comportamiento

- **Modelan clases:** \* Diagrama de máquina de estados.
  - Diagrama de CU.
  - Diagrama de actividad.
- **Modelan interacciones:** \* Diagrama de secuencia.
  - Diagrama de comunicaciones.
  - Diagrama de timing.
  - Diagrama de descripción de interacción.

# Workflow de Requerimientos

Se centra en el dominio del problema, usa lenguaje natural, ofrece una vista externa del sistema. Tiene como entrada las necesidades del cliente y tiene como salida (artefactos):

- **a) MODELO DE CASOS DE USO:** Para eso implementa como herramienta al diagrama de casos de uso (que muestra la interacción entre los actores y las funcionalidades). Como la vista gráfica no es suficiente también implementa descripciones de casos de uso y descripciones de actores. Es la salida más importante.
- **b) Modelo de dominio:** Este modelo implementa como herramienta al diagrama de clases pero enfocándose estrictamente en el negocio (las clases de entidad).
- **c) Descripción de interfaces de usuario.**
- **d) Descripción de la arquitectura.**

## Workflow de Análisis

Establece un puente entre “el mundo del usuario, dominio del problema” y “el mundo del desarrollador, dominio de la solución”. Utiliza lenguaje técnico, ofrece una vista interna del sistema, nos permite realizar una especificación más precisa de los requerimientos.

Tiene como entrada los modelos del workflow de requerimientos y tiene como salida:

### a) Las clases de análisis

Son clases de fabricación pura, son las abstracciones lógicas del sistema, pertenecen al dominio de la solución:

- **Clases de entidad:** Representan la información y el comportamiento de conceptos o fenómenos del negocio. Suelen derivarse de las clases de entidad del negocio. Tienen una vida larga y son persistentes.
- **Clases de control:** Representan la coordinación, secuencia, transacciones y control de otros objetos. Encapsulan la lógica de un caso de uso en concreto. Además representan cálculos complejos y derivaciones que no pueden asignarse a una clase de entidad en específico. Funcionan de nexo entre las clases de Entidad y las clases boundary, ya que estas no pueden comunicarse directamente. No son persistente, su vida acaba cuando finaliza la ejecución del flujo.
- **Clases boundary:** Representan la interacción entre los actores y el sistema. Sirven de medio para que el actor pueda ingresar información y ver los resultados y el sistema pueda recibir la información y presentar los resultados.

### Otras salidas del workflow:

- **b) Las realizaciones de caso de uso:** Implementa como herramienta el diagrama de casos de uso o el diagrama de comunicaciones. Muestran el paso a paso de como las instancias de las clases de análisis colaboran e intercambian mensajes para llevar a cabo el flujo de un caso de uso. Es la salida más importante.

- **c) Paquete de análisis:** Agrupaciones de clases de análisis que están fuertemente relacionadas.
- **d) Descripción de la arquitectura.**

## Dominios del Desarrollo

### Dominio del Problema

Es el entorno o espacio del negocio del cliente donde existen las necesidades, desafíos y procesos propios de la organización. En este dominio no se aborda nada de la lógica interna ni técnica del sistema de información, sino que se enfoca estrictamente en el negocio.

- **Objetivo:** Permite obtener "qué" se necesita resolver.
- **Relación con el PUD:** Está directamente vinculado con el Workflow de Requerimientos.

### Dominio de la Solución

Es el espacio técnico y de software donde las necesidades, desafíos y procesos del negocio identificados en el dominio del problema se transforman en estructuras lógicas, algoritmos y componentes de software para construir el sistema de información.

- **Objetivo:** Su enfoque principal es el "cómo" se va a implementar y construir técnica y físicamente el software.
- **Relación con el PUD:** Está directamente vinculado con los Workflows de Diseño, Implementación y Prueba.

## Visibilidad entre Objetos

La visibilidad es la capacidad de un objeto de ver a otro para que pueda enviarle un mensaje, en la estructura del sistema, esto determina el nivel de acoplamiento entre las clases. Existen dos tipos de visibilidad:

- **Visibilidad de atributos (permanente):** Esta se da cuando un objeto tiene una referencia directa y duradera hacia otro objeto a través de un atributo de su clase, estas son a largo plazo (mientras el objeto exista, tendrá acceso al otro objeto vinculado). Estas se modelan a través de las relaciones de asociación, agregación o composición.
- **Visibilidad de parámetros (momentánea):** Se produce cuando un objeto recibe la referencia de otro objeto únicamente durante la ejecución de un método, generalmente como un argumento o parámetro. En este caso se modelan gracias a las dependencias, proporcionan un menor acoplamiento ya que la conexión es "transitoria" (dura lo que dura la ejecución de ese método).

## Patrones y Responsabilidades

Un patrón es una solución genérica y reutilizable para un problema común dentro de cierto contexto determinado, es una plantilla o guía de cómo resolver un problema que puede darse en muchas situaciones.

En sí dentro del “WORKFLOW” de análisis los implementamos para plantear a qué clase (Boundary, Controller o Entity) le corresponden ciertas responsabilidades y además me propician que el modelo resultante tenga alta cohesión y bajo acoplamiento.

- **Responsabilidades del HACER:** Se refieren a las acciones que un objeto debe ser capaz de ejecutar (son sus métodos), en sí puede ser algo que haga el mismo, iniciar una acción en otros objetos, controlar y coordinar actividades en otros objetos y/o crear otros objetos.
- **Responsabilidades del CONOCER:** Se refiere a la información que un objeto debe de mantener y ser capaz de proporcionar (atributos y/o relaciones con otros objetos).

## Patrones GRASP

Son un conjunto de directrices fundamentales para asignar responsabilidades a clases y objetos, estos patrones se aplican intensamente durante el Workflow de Análisis para actuar las clases necesarias y distribuir las responsabilidades para cumplir con los requerimientos funcionales. Se tienen en cuenta 5:

- **Patrón Experto:** Establece que se debe de asignar una responsabilidad a la clase que cuenta con la información necesaria para llevarla a cabo.
- **Patrón Creador:** Determina qué clase debe crear una nueva instancia de otra. Una clase A debe crear una instancia de otra clase B si: Tiene o agrega a B, tiene los datos de inicialización de B.
- **Patrón Controlador:** Asigna la responsabilidad de manejar los eventos del sistema a una clase que representa el escenario de caso de uso. Es una Fabricación Pura que coordina la lógica, en sí asegura que los procesos del dominio sean manejados por la capa del dominio y no por la de interfaz.
- **Patrón Bajo Acoplamiento:** Se busca asignar responsabilidades para mantener un bajo acoplamiento, mientras más independientes sean las clases se reduce el impacto de los cambios y son más reutilizables.
- **Patrón Alta Cohesión:** Se busca asignar una responsabilidad a la clase de manera que tenga un número relativamente pequeño de operaciones con funcionalidad relacionada y poco trabajo por hacer. Por esto colabora con otros objetos para compartir esfuerzo si la tarea es grande.